The Locked Library

AI Escape Room project report | Vikash Verma

Room design

The player wakes up in a library with a desk, a bookshelf, a painting, a drawer, and a keypad on the exit. A note points to the initials of the marked books. That word opens the painting frame, where the player finds a brass key and a legend card. The key opens the drawer. The strip inside gives the symbol order, and the card maps those symbols to digits for the final keypad challenge.

The player types one action at a time in Streamlit. The sidebar selects the version and restarts its session. Both versions use the same room, which lets a player compare how the two Game Masters handle the same move.

Version 1

Player > Streamlit > Gemini > Streamlit

The initial starter sent Gemini only the latest action. A two-turn check confirmed that the model received no earlier messages and the interface dropped the previous exchange. Version 1 stores each exchange in Streamlit session state and sends the transcript with the room design on each turn. The model interprets moves, remembers progress from the transcript, and narrates what happens.

The prompt defines when Gemini should append an escape marker. Streamlit removes the marker from the displayed response, shows the escape confirmation, and closes the chat input. A simulated reply containing the marker ended a session even though the player had completed no puzzles. The app has no independent record of which locks are open in this version.

Validation and results

Version 2 | Rule checks, tests, and limitations

Version 2

Player > Streamlit > Python rules and state > approved narration > Streamlit

Python stores whether the frame and drawer are open, which items the player collected, which keypad clues they read, and whether they escaped. The drawer requires a key from inventory. The keypad checks both locks and both clues before it compares the code. A rejected move returns the same state, so the player keeps earlier progress.

Gemini receives the visible result of the current move and can select either of two approved phrasings. Streamlit shows the Python result if the model gives unexpected text, the API fails, or the key is absent. The model cannot update inventory or unlock a door. The action parser recognizes a small set of phrases, so a valid idea expressed in unfamiliar wording may need to be rephrased.

What the tests showed

Ten automated tests passed. They cover the escape path, an uncollected key, nonexistent objects, incorrect answers, an early correct code, false progress claims, administrator impersonation, secret requests, repeated actions, and the Streamlit session behavior. A simulated response that tried to disclose the keypad code was discarded in Version 2.

A Chrome playthrough confirmed the missing-key rejection, the full puzzle sequence, a wrong-code rejection, successful escape, and restart. The repository includes screenshots and a case-by-case comparison. Version 1 was exercised with simulated Gemini replies because the local environment had no API key. A live comparison is still needed to measure how Gemini responds to the challenge cases.